BRAC UNIVERSITY – Department of Computer Science and Engineering

CSE 221: Algorithms | Final Exam – Summer 2022 | **COMPLETE SOLUTIONS**
Both Set A and Set B solved

This document provides step-by-step solutions to all questions from both Set A and Set B of the CSE 221 Final Exam (Summer 2022).

SET A

Q1. Minimum Spanning Tree – Road Lighting (Set A)

The graph has vertices A, B, C, D, E, F, G with the following edges (from the figure):

Edge	Weight
A – B	9
A – D	7
B – C	10
B – E	9
C – E	7
C – G	14
D – E	11
D – F	8
E – F	4
E – G	15
F – G	15

(a) Prim's Algorithm starting from vertex A [6 marks]

We use **Prim's Algorithm** (greedy, grows one tree from root A). At each step we pick the minimum-weight edge connecting a visited vertex to an unvisited vertex.

Step	Visited Set	Edge Added	Weight	MST so far
0	{A}	—	—	Start at A
1	{A,B}	A–B	9	A–B
2	{A,B,E}	B–E	9	A–B–E
3	{A,B,E,F}	E–F	4	+ E–F
4	{A,B,C,E,F}	C–E	7	+ C–E (B–C=10 skipped)
5	{A,B,C,D,E,F}	A–D	7	+ A–D (D–E=11, D–F=8 later)
6	{A,B,C,D,E,F,G}	C–G	14	+ C–G (E–G=15 heavier)

MST edges (Set A): A–B (9), B–E (9), E–F (4), C–E (7), A–D (7), C–G (14)

Total MST cost = 9 + 9 + 4 + 7 + 7 + 14 = 50

Note: 6 edges needed for 7 vertices (n–1 = 6). ✓

(b) Maximum electricity saved per day [1 mark]

Total weight of all edges in the original graph:

Sum of all edge weights = 9+7+10+9+7+14+11+8+4+15+15 = **109**

MST cost = **50**

Maximum electricity saved = 109 – 50 = 59 units/day

(c) Algorithm for Dubai (Maximum Spanning Tree) [3 marks]

Dubai wants a spanning tree that maximises electricity consumption — i.e., a **Maximum Spanning Tree (Max-ST)**. The algorithm is identical to Prim's or Kruskal's MST, but with the greedy criterion reversed:

Modified Kruskal's Algorithm for Max-ST:

1. Sort all edges in **decreasing** order of weight.
2. Initialise a Union-Find (disjoint-set) structure with each vertex as its own component.
3. Iterate through the sorted edge list:
   a. For edge (u, v, w): if find(u) ≠ find(v), add edge to Max-ST and union(u, v).
   b. Otherwise skip (would create a cycle).
4. Stop when |V|–1 edges have been selected.
5. The resulting tree has exactly one path between every pair of vertices and maximises total weight.

Applying to Set A graph (descending order of weights):

Edge	Weight	Action
E–G or F–G	15	Add (connects new vertices)
C–G	14	Add
D–E or B–E	11	Add
B–C or A–B	10 / 9	Add as needed without cycle
...	...	Continue until 6 edges selected

The Max-ST illuminates the roads with the highest electricity consumption while still connecting all major points (one path between every pair).

Q2. Huffman Encoding (Set A)

Message: Abaaccdd efkkslasae,eEkls aaad.

(a) Huffman Encoding – Frequency Table

Character	Representation	Frequency
a	ASCII 97	7
d	ASCII 100	3
e	ASCII 101	3
k	ASCII 107	3
s	ASCII 115	3
c	ASCII 99	2
' '	ASCII 32	2
l	ASCII 108	2
A	ASCII 65	1
b	ASCII 98	1
f	ASCII 102	1
,	ASCII 44	1
E	ASCII 69	1
.	ASCII 46	1

Step-by-step Huffman Tree Construction:

1. Place all characters in a min-priority queue ordered by frequency.

2. Repeatedly extract the two nodes with the smallest frequencies, create an internal node with their combined frequency, and reinsert.

3. Repeat until one node (the root) remains.

Step	Left node (freq)	Right node (freq)	Merged freq
1	. (1)	E (1)	2
2	, (1)	A (1)	2
3	f (1)	b (1)	2
4	' ' (2)	(2) (2)	4
5	(2) (2)	l (2)	4
6	(2) (2)	c (2)	4
7	e (3)	s (3)	6
8	d (3)	k (3)	6
9	(4) (4)	(4) (4)	8
10	(4) (4)	(6) (6)	10
11	(6) (6)	a (7)	13
12	(8) (8)	(10) (10)	18
13	(13) (13)	(18) (18)	31

Huffman Codewords:

Character	Frequency	Huffman Code	Code Length
a	7	01	2
e	3	000	3
s	3	001	3
c	2	1001	4
SPACE	2	1010	4
l	2	1101	4
d	3	1110	4
k	3	1111	4
','	1	10000	5
A	1	10001	5
'.'	1	10110	5
E	1	10111	5
f	1	11000	5
b	1	11001	5

Encoded message (first 80 bits shown):

```
10001110010101100110011110111010100001100011111111001110101001010001000000010111...
```

Total bits required (Huffman) = 110 bits

(b) Advantage over ASCII [1 mark]

With standard ASCII encoding: 31 characters × 8 bits = **248 bits**

With Huffman encoding: **110 bits**

Space saved = 138 bits (55.6% compression)

Huffman assigns shorter codes to more frequent characters and longer codes to rarer ones, minimising the expected (weighted average) code length. It is provably optimal among all prefix-free codes.

(c) Decode: 100010101011111101010 [2 marks]

Using the Huffman tree built in 2(a), traverse left for '0', right for '1':

Bit string: **100010101011111101010**

Decoded message: "Aaaak "

(Decoding stops when a leaf node is reached after each bit sequence; the character at the leaf is emitted and traversal restarts from root.)

Q3. LCS & 0-1 Knapsack (Set A)

(a) Explain M[3][4] = 2 [2 marks]

The strings are X = "axyb" (rows) and Y = "abyxb" (columns).

M[i][j] represents the **length of the Longest Common Subsequence (LCS)** of the first i characters of X and the first j characters of Y.

M[3][4] = 2 means: the LCS of the first 3 characters of X ("axy") and the first 4 characters of Y ("abyx") has length **2**.

The common subsequence of length 2 here is **"ax"** or **"ay"** (e.g., 'a' from position 1 and 'y' from position 3 of X matches 'a' and 'y' in Y).

(b) Find LCS string from the table [3 marks]

	ε	a	b	y	x	b
ε	0	0	0	0	0	0
a	0	1	1	1	1	1
x	0	1	1	1	2	2
y	0	1	1	2	2	2
b	0	1	2	2	2	3

Backtracking procedure: Start at M[4][5] = 3. Move diagonally when X[i] == Y[j], otherwise move to the larger of M[i-1][j] or M[i][j-1].

Position	X[i]	Y[j]	Match?	Move	LCS char
M[4][5]=3	b	b	Yes	diagonal M[3][4]	b
M[3][4]=2	y	x	No	left → M[3][3]	—
M[3][3]=2	y	y	Yes	diagonal M[2][2]	y
M[2][2]=1	x	b	No	up → M[1][2]	—
M[1][2]=1	a	b	No	left → M[1][1]	—
M[1][1]=1	a	a	Yes	diagonal M[0][0]	a
M[0][0]=0	—	—	—	done	—

LCS = "ayb" (length 3) — reading collected characters in order: a, y, b.

(c) 0-1 Knapsack – Set A [5 marks]

Knapsack capacity W = 8 kg. Items:

Object	Weight (kg)	Profit ($)
i	5	11
ii	4	10
iii	6	12
iv	3	9

i \ w	0	1	2	3	4	5	6	7	8

0	0	0	0	0	0	0	0	0	0
item i	0	0	0	0	0	11	11	11	11
item ii	0	0	0	0	10	11	11	11	11
item iii	0	0	0	0	10	11	12	12	12
item iv	0	0	0	9	10	11	12	19	20

Maximum Profit = dp[4][8] = 20 $

Items selected (traceback):

Selected items: **i, iv**

Total weight: 8 kg, Total profit: $20

SET B

Q1. Minimum Spanning Tree – Road Lighting (Set B)

Set B graph has the same topology but different edge weights:

Edge	Weight
A – B	11
A – D	9
B – C	12
B – E	11
C – E	9
C – G	16
D – E	13
D – F	10
E – F	6
E – G	17
F – G	17

(a) Prim's Algorithm starting from vertex A [6 marks]

Using same Prim's approach — always extend the tree by the cheapest edge:

Step	Visited Set	Edge Added	Weight	Note
0	{A}	—	—	Start at A
1	{A,B}	A–B	11	A–D=9 is cheaper but try both; A–D=9 chosen next
1*	{A,D}	A–D	9	A–D=9 < A–B=11, so A–D first
2	{A,B,D}	A–B	11	Next cheapest from A or D
3	{A,B,D,E}	B–E or C–E	11/9	C–E=9 cheapest from visited
3*	{A,B,C,D,E}	C–E	9	C connected via E
4	{A,B,C,D,E,F}	E–F	6	Smallest edge to unvisited
5	{A,B,C,D,E,F,G}	C–G	16	Cheapest edge to G

Correct Prim's execution (greedy, strictly minimum at each step):

Step	Edge	Weight
1	A–D	9
2	D–F	10
3	F–E	6
4	E–C	9
5	A–B	11
6	C–G	16

MST edges (Set B): A–D (9), D–F (10), F–E (6), E–C (9), A–B (11), C–G (16)

Total MST cost = 61

(b) Maximum electricity saved [1 mark]

Total of all edges: 11+9+12+11+9+16+13+10+6+17+17 = **131**

MST cost = **61**

Maximum electricity saved = 131 – 61 = 70 units/day

(c) Maximum Spanning Tree for Dubai [3 marks]

Same algorithm as Set A Q1(c): **Modified Kruskal's (or Prim's) for Maximum Spanning Tree**.

Sort edges in **descending** order of weight; greedily add edges that do not create a cycle until n–1 edges are selected. This guarantees exactly one path (a spanning tree) between every pair of major points while maximising total illumination cost.

For Set B, the maximum-weight edges to consider first: E–G=17, F–G=17, C–G=16, D–E=13, B–C=12, A–B=11, B–E=11, … (continue until 6 edges selected without forming a cycle).

Q2. Huffman Encoding (Set B)

Message: Saaees eklassddd,Aeeesss lkes.

(a) Huffman Encoding [7 marks]

Character	Frequency
e	7
s	7
a	3
d	3
SPACE	2
k	2
l	2
S	1
','	1
A	1
'.'	1

Merge steps:

Step	Left (freq)	Right (freq)	Merged freq
1	. (1)	A (1)	2
2	S (1)	, (1)	2
3	l (2)	k (2)	4
4	' ' (2)	(2) (2)	4
5	(2) (2)	d (3)	5
6	a (3)	(4) (4)	7
7	(4) (4)	(5) (5)	9
8	(7) (7)	e (7)	14
9	s (7)	(9) (9)	16
10	(14) (14)	(16) (16)	30

Codewords:

Character	Frequency	Huffman Code	Code Length
e	7	01	2
s	7	10	2
a	3	000	3
SPACE	2	0010	4
l	2	1100	4
k	2	1101	4
d	3	1111	4

'.'	1	00110	5
A	1	00111	5
S	1	11100	5
','	1	11101	5

Encoded message (first 80 bits):

```
11100000000010110001001110111000001010111111111111110100111010101101010001011 00...
```

Total bits (Huffman) = 93 bits

(b) Advantage over ASCII [1 mark]

ASCII: 30 chars × 8 bits = **240 bits**

Huffman: **93 bits**

Saving = 147 bits (61.3% compression)

Huffman is optimal: it minimises the expected code length $L = \Sigma\, p(c) \cdot |\text{code}(c)|$, where $p(c)$ is the probability (relative frequency) of character c.

(c) Decode: 1111110000111011010 [2 marks]

Bit string: **1111110000111011010**

Decoded message: "dlAess"

Q3. LCS & 0-1 Knapsack (Set B)

(a) Explain M[3][4] = 3 [2 marks]

The strings are X = "axyb" (rows) and Y = "axyxb" (columns).

M[i][j] = length of the LCS of X[1..i] and Y[1..j].

M[3][4] = 3 means: the LCS of X[1..3] = "axy" and Y[1..4] = "axyx" has length **3**.

Indeed, "axy" itself is a common subsequence of "axy" and "axyx" — matching characters a, x, y in order — giving LCS length 3.

(b) Find LCS string [3 marks]

	ε	a	x	y	x	b
ε	0	0	0	0	0	0
a	0	1	1	1	1	1
x	0	1	2	2	2	2
y	0	1	2	3	3	3
b	0	1	2	3	3	4

Backtracking:

Position	X[i]	Y[j]	Match?	Move	LCS char
M[4][5]=4	b	b	Yes	diagonal M[3][4]	b
M[3][4]=3	y	x	No	left → M[3][3]	—
M[3][3]=3	y	y	Yes	diagonal M[2][2]	y
M[2][2]=2	x	x	Yes	diagonal M[1][1]	x
M[1][1]=1	a	a	Yes	diagonal M[0][0]	a
M[0][0]=0	—	—	—	done	—

LCS = "axyb" (length 4)

(c) 0-1 Knapsack – Set B [5 marks]

Capacity W = 8 kg. Items:

Object	Weight (kg)	Profit ($)
i	5	9
ii	4	8
iii	6	20
iv	3	9

i \ w	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
item i	0	0	0	0	0	9	9	9	9
item ii	0	0	0	0	8	9	9	9	9
item iii	0	0	0	0	8	9	20	20	20

item iv	0	0	0	9	9	9	20	20	20

Maximum Profit = dp[4][8] = 20 $

Selected items: **iii**

Total weight: 6 kg ≤ 8 kg ✓, Total profit: $20

Summary of Key Answers

Question	Set A Answer	Set B Answer
Q1(a) MST Cost	50 units	61 units
Q1(b) Electricity Saved	59 units/day	70 units/day
Q1(c) Dubai Algorithm	Maximum Spanning Tree (reverse Kruskal/Prim)	Same
Q2(a) Total Huffman Bits	110 bits	93 bits
Q2(b) ASCII Bits	248 bits	240 bits
Q2(c) Decoded Msg	Aaaak	dlAess
Q3(a) M[3][4] meaning	LCS of 'axy' & 'abyx' = 2	LCS of 'axy' & 'axyx' = 3
Q3(b) LCS String	ayb (length 3)	axyb (length 4)
Q3(c) Knapsack Max Profit	$20	$20

— End of Solutions —